

# Homework for Spring AI

## 1. The Sarcastic Poet

- **Task:** Expose an endpoint `GET /poem?topic={t}&lang={l}`.
- **Requirement:** Use `ChatClient` with a fluent API.
- **Structured Output:** Return a JSON object (Java Record) with fields: `title`, `poem_text`, `rhyme_scheme` (e.g., AABB).
- **Configuration:** Configure the application to run on **Ollama** (local) by default, but allow switching to **OpenAI** just by changing a profile `application-openai.yml`.

## 2. The "Vibe" Playlist Matcher (Embeddings & Vector Search)

- **Data:** Create a list of 10 distinct song lyrics (or summaries of their meaning) in code.
- **Ingest:** Embed them and store them in **PGVector**.
- **Search:** Create an endpoint `GET /match-vibe?feeling={text}`.
  - *Input:* "I am feeling sad and want to give up."
  - *Expected Result:* Returns the song "Fix You" or similar (even if the word "sad" isn't in the lyrics).
- **Filter:** Add metadata for `genre` (Rock, Pop) and filter results to only show "Rock" songs.

## 3. The "Employee Handbook" Bot (Basic RAG)

- **Data:** Create a dummy PDF named `policy.pdf` containing rules like "Remote work is allowed on Fridays" and "Expense limit is \$50".
- **ETL:** Write a `CommandLineRunner` that reads the PDF using `PagePdfDocumentReader`, splits it into tokens, and loads it into the Vector Store on startup.

- **Chat:** Create an endpoint where users can ask "Can I work from home on Monday?".
- **Success Criteria:** The bot must correctly answer "No, only on Fridays" citing the document.

#### 4. The "Safe & Forgetful" Assistant (Advisors)

- **Memory:** Implement a chat endpoint that remembers your name.
  - *User:* "Hi, I'm Alice."
  - *User:* "What is my name?" → *Bot:* "Alice."
  - *Constraint:* Use `MessageWindowChatMemory` with a max of **3 messages**. Prove that after 4 exchanges, it forgets the first message.
- **Safety:** Add a `SafeGuardAdvisor` that blocks the word "competitor".
- **Logging:** Enable `SimpleLoggerAdvisor` and verify in your console that you can see the conversation history being injected into the prompt.

#### 5. The "Stock Trader" Agent (Tool Calling)

- **Tools:** Create two `@Tool` methods:
  - `getStockPrice(String ticker)` : Returns a random double (e.g., 150.0).
  - `buyStock(String ticker, int quantity)` : Returns "Bought X shares of Y".
- **Agent:** Build a `ChatClient` that has access to both tools.
- **Scenario:**
  - *User:* "If Apple is under \$200, buy me 10 shares."
  - *Logic:* The LLM should first call `getStockPrice("AAPL")`. If the result is < 200, it should automatically call `buyStock`.
  - *Output:* Final response should be "I checked the price (it was \$150) and bought 10 shares for you."

## 6. The "Customer Support Agent" (Tools + RAG + Advisors)

- **RAG:** Ingest a "User Manual" PDF. The bot answers "How do I reset my router?" using this.
- **Tools:** If the user says "My internet is down, please restart it", the bot calls a `rebootRouter(String serialNo)` tool.
- **Flow:**
  - User: "How do I turn it on?" → **RAG** answers.
  - User: "It's not working, please restart it." → **Tool** executes.